

实验一：比特币测试网交易与区块解析

实验目的：

在比特币测试网（Testnet）上生成钱包并发送一笔交易；

获取该交易的原始十六进制数据，并逐字节解析到每个比特位；

获取一个完整区块的原始数据，解析区块头，并验证工作量证明（PoW）；

解析交易的解锁脚本和锁定脚本，翻译每个操作码的含义。

实验环境

操作系统：Ubuntu 20.04

Python 版本：3.12.4

主要Python库：bit、requests

网络：比特币测试网

区块浏览器API：<https://blockstream.info/testnet/>

实验步骤：

安装必要库：

代码块

```
1 pip install bit requests -i https://mirrors.aliyun.com/pypi/simple/
```

```
(base) ubuntu@ubuntu:~/Desktop$ pip install bit requests -i https://mirrors.aliyun.com/pypi/simple/
Looking in indexes: https://mirrors.aliyun.com/pypi/simple/
Collecting bit
  Downloading https://mirrors.aliyun.com/pypi/packages/3d/3a/188818b767afc4ebad66c3bd30205fdea137d6c7dd73a4767dc00dd12fbf/bit-0.8.0-py3-none-any.whl (68 kB)
    68.9/68.9 kB 413.6 kB/s eta 0:00:00
Requirement already satisfied: requests in /home/ubuntu/anaconda3/lib/python3.12/site-packages (2.32.2)
Collecting coincurve>=4.3.0 (from bit)
  Downloading https://mirrors.aliyun.com/pypi/packages/b7/75/050b6fd08978de85a7b480f0f220ab6a30967c0910119f3096a8dd40befc/coincurve-21.0.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.6 MB)
    1.6/1.6 MB 262.1 kB/s eta 0:00:00
Requirement already satisfied: charset-normalizer<4,>=2 in /home/ubuntu/anaconda3/lib/python3.12/site-packages (from requests) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in /home/ubuntu/anaconda3/lib/python3.12/site-packages (from requests) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/ubuntu/anaconda3/lib/python3.12/site-packages (from requests) (2.2.2)
Requirement already satisfied: certifi>=2017.4.17 in /home/ubuntu/anaconda3/lib/python3.12/site-packages (from requests) (2024.6.2)
Installing collected packages: coincurve, bit
Successfully installed bit-0.8.0 coincurve-21.0.0
```

创建工作文件：

代码块

```
1  mkdir -p ~/Desktop/bitcoin_homework
```

生成测试网钱包：

代码块

```
1  nano create_wallet.py
```

代码块

```
1  from bit import PrivateKeyTestnet
2
3  key = PrivateKeyTestnet()
4  print("测试网地址:", key.address)
5  print("私钥 WIF:", key.to_wif())
```

然后用它生成我的测试网地址：

(base) ubuntu@ubuntu: ~/Desktop/bitcoin_homework\$ python3 create_wallet.py
测试网地址: mvP7ogEXJGs7paKQu5pfRcegWampataPgx

代码块

```
1 mvP7ogEXJGs7paKQu5pfRcegWampataPgx
2
```

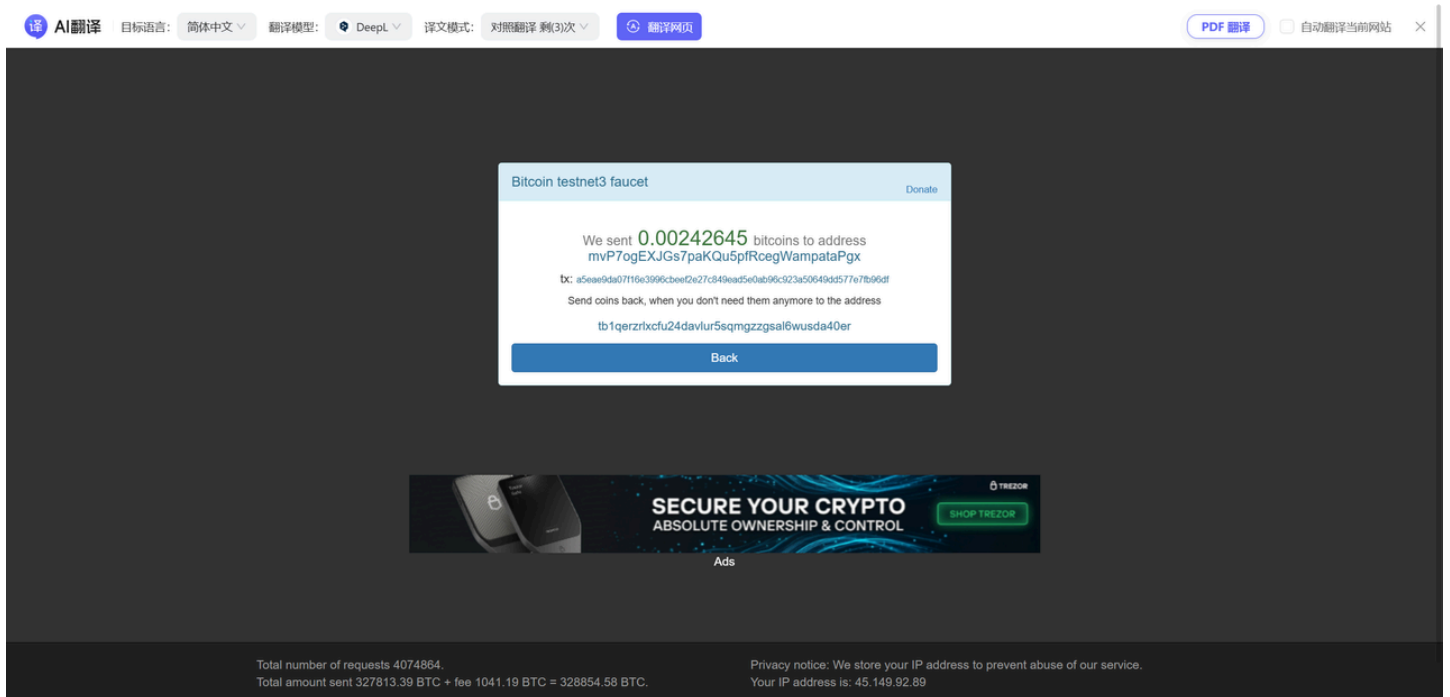
以及私钥WIF:

cRCwpEbhyqaTum6qNsXYiKZEwPhG37oxYBmaQakw6WzJx1kJQmC

代码块

```
1 cRCwpEbhyqaTum6qNsXYiKZEwPhG37oxYBmaQakw6WzJx1kJQmC
```

然后访问水龙头 <https://coinfaucet.eu/en/btc-testnet/>, 领取测试币:



接下来发送测试网交易:
创建manual_send.py

代码块

```
1 from bit import PrivateKeyTestnet
2 import requests
3
```

```

4  wif = 'cUtjujEzZ7n4AAPGqpfnK93ozHhP3X4mdfeTX5XWbLmpPcYpZCs'
5  key = PrivateKeyTestnet(wif)
6  to_address = 'mvP7ogEXJGs7paKQu5pfRcegWampataPgx'
7
8
9  tx_hex = key.create_transaction([(to_address, 5000, 'satoshi')], fee=200)
10 print("交易十六进制:", tx_hex)
11
12 # 广播到 blockstream.info
13 url = 'https://blockstream.info/testnet/api/tx'
14 response = requests.post(url, data=tx_hex, headers={'Content-Type':
15 'text/plain'})
16
17 if response.status_code == 200:
18     txid = response.text.strip()
19     print("广播成功! 交易ID:", txid)
20 else:
21     print("广播失败, 状态码:", response.status_code)
22     print("错误信息:", response.text)

```

```

(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$ python3 manual_send.py
交易十六进制 (已签名): 010000000104fd126fc9ef9a6b1bb673770e20724d28953ebd2f12985
946e11a90e68e4678010000006a47304402206940511f4d205bade221ebbc693b946d5f6a56d89d2
29ccdf3f5eb7ffc2cc366022077d8e62b80f9f2a45df66fe1410a46de6405a54aee0d64b0bd047f2
a3c1d094e012103ccdc2b75b397f644dad9553dafa506829c85db45dbbfef65ca5caafad2e1e17cf
ffffffff0288130000000000001976a914a30e7f8c2e439d8e83b1ecd8f8a97dfbc67c90c588aca4d
70000000000000001976a914759c1790f092a213dd2e2ef2cbc4ec1544a1e9e688ac00000000

```

代码块

```

1  010000000104fd126fc9ef9a6b1bb673770e20724d28953ebd2f12985946e11a90e68e467801000
0006a47304402206940511f4d205bade221ebbc693b946d5f6a56d89d229ccdf3f5eb7ffc2cc366
022077d8e62b80f9f2a45df66fe1410a46de6405a54aee0d64b0bd047f2a3c1d094e012103ccdc2
b75b397f644dad9553dafa506829c85db45dbbfef65ca5caafad2e1e17cffffffff028813000000
0000001976a914a30e7f8c2e439d8e83b1ecd8f8a97dfbc67c90c588aca4d700000000000001976a
914759c1790f092a213dd2e2ef2cbc4ec1544a1e9e688ac00000000

```

```

广播成功! 交易ID: 02d79bfb36b4a10db70bb181bf823afdcae7b6bd7049a3c79397b069d3a

```

代码块

```

1  02d79bfb36b4a10db70bb181bf823afdcae7b6bd7049a3c79397b069d3ace6d0

```

接下来获取交易原始十六进制：

创建 get_raw_tx.py：

代码块

```
1  import requests
2
3  txid = '02d79bfb36b4a10db70bb181bf823afdcae7b6bd7049a3c79397b069d3ace6d0'
4  url = f'https://blockstream.info/testnet/api/tx/{txid}/hex'
5  response = requests.get(url)
6  raw_hex = response.text.strip()
7  print("原始交易十六进制:")
8  print(raw_hex)
9  with open('raw_tx.hex', 'w') as f:
10     f.write(raw_hex)
11  print("已保存到 raw_tx.hex")
12
```

```
(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$ nano get_raw_tx.py
(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$ python3 get_raw_tx.py
原始交易十六进制:
010000000104fd126fc9ef9a6b1bb673770e20724d28953ebd2f12985946e11a90e68e4678010000
006a47304402206940511f4d205bade221ebbc693b946d5f6a56d89d229ccdf3f5eb7ffc2cc36602
2077d8e62b80f9f2a45df66fe1410a46de6405a54aee0d64b0bd047f2a3c1d094e012103ccdc2b75
b397f644dad9553dafa506829c85db45dbbfef65ca5caafad2e1e17cfffffffff0288130000000000
001976a914a30e7f8c2e439d8e83b1ecd8f8a97dfbc67c90c588aca4d70000000000001976a91475
9c1790f092a213dd2e2ef2cbc4ec1544a1e9e688ac00000000
已保存到 raw_tx.hex
```

代码块

```
1  010000000104fd126fc9ef9a6b1bb673770e20724d28953ebd2f12985946e11a90e68e467801000
0006a47304402206940511f4d205bade221ebbc693b946d5f6a56d89d229ccdf3f5eb7ffc2cc366
022077d8e62b80f9f2a45df66fe1410a46de6405a54aee0d64b0bd047f2a3c1d094e012103ccdc2
b75b397f644dad9553dafa506829c85db45dbbfef65ca5caafad2e1e17cfffffffff028813000000
00000001976a914a30e7f8c2e439d8e83b1ecd8f8a97dfbc67c90c588aca4d70000000000001976a
914759c1790f092a213dd2e2ef2cbc4ec1544a1e9e688ac00000000
```

然后逐字节解析交易：

创建 parse_tx.py

代码块

```
1  import sys
2  from binascii import unhexlify
3
4  def read_varint(data, offset):
5      first = data[offset]
6      if first < 0xFD:
7          return first, 1
8      elif first == 0xFD:
9          return int.from_bytes(data[offset+1:offset+3], 'little'), 3
10     elif first == 0xFE:
11         return int.from_bytes(data[offset+1:offset+5], 'little'), 5
12     elif first == 0xFF:
13         return int.from_bytes(data[offset+1:offset+9], 'little'), 9
14     else:
15         raise ValueError("Invalid VarInt")
16
17 def byte_to_bin(b):
18     """将单个字节转为8位二进制字符串"""
19     return bin(b)[2:].zfill(8)
20
21 def print_bytes_hex_and_bin(data, start, length, label):
22     """打印指定范围的每个字节的十六进制和二进制"""
23     print(f" {label} (长度 {length} 字节):")
24     for i in range(length):
25         b = data[start+i]
26         print(f"    [{i:2d}] 0x{b:02x} {byte_to_bin(b)}")
27     print()
28
29 def parse_transaction(raw_hex):
30     data = bytes.fromhex(raw_hex)
31     offset = 0
32     print("交易原始数据解析")
33     print(f"总字节数: {len(data)}\n")
34
35     # 1. 版本号
36     version = int.from_bytes(data[offset:offset+4], 'little')
37     print(f"1. 版本号 (Version): {version} (0x{version:08x})")
38     print_bytes_hex_and_bin(data, offset, 4, "版本号字节")
39     offset += 4
40
```

```

41     # 2. 输入计数
42     in_count, v_size = read_varint(data, offset)
43     print(f"2. 输入计数 (Input Count): {in_count}")
44     print_bytes_hex_and_bin(data, offset, v_size, "输入计数 VarInt 字节")
45     offset += v_size
46
47     # 3. 解析每个输入
48     for i in range(in_count):
49         print(f"\n--- 输入 #{i+1} ---")
50         # 前序交易哈希
51         prev_hash_bytes = data[offset:offset+32]
52         prev_hash_hex = prev_hash_bytes[::-1].hex()
53         print(f" 前序交易哈希 (Previous Tx Hash): {prev_hash_hex}")
54         print_bytes_hex_and_bin(data, offset, 32, f"输入#{i+1} 前序哈希字节")
55         offset += 32
56
57         # 输出索引
58         prev_index = int.from_bytes(data[offset:offset+4], 'little')
59         print(f" 前序输出索引 (Previous Index): {prev_index}")
60         print_bytes_hex_and_bin(data, offset, 4, f"输入#{i+1} 索引字节")
61         offset += 4
62
63         # 解锁脚本长度
64         script_len, s_len = read_varint(data, offset)
65         print(f" 解锁脚本长度 (ScriptSig Length): {script_len}")
66         print_bytes_hex_and_bin(data, offset, s_len, f"输入#{i+1} 脚本长度
VarInt")
67         offset += s_len
68
69         # 解锁脚本
70         script_sig = data[offset:offset+script_len]
71         print(f" 解锁脚本 (ScriptSig): {script_sig.hex()}")
72         if script_len > 0:
73             print_bytes_hex_and_bin(data, offset, script_len, f"输入#{i+1} 解锁
脚本字节")
74         else:
75             print("  (空脚本)")
76         offset += script_len
77
78         # 序列号
79         seq = int.from_bytes(data[offset:offset+4], 'little')
80         print(f" 序列号 (Sequence): {seq}")
81         print_bytes_hex_and_bin(data, offset, 4, f"输入#{i+1} 序列号字节")
82         offset += 4
83
84     # 4. 输出计数
85     out_count, v_size = read_varint(data, offset)

```

```

86     print(f"\n3. 输出计数 (Output Count): {out_count}")
87     print_bytes_hex_and_bin(data, offset, v_size, "输出计数 VarInt 字节")
88     offset += v_size
89
90     # 5. 解析每个输出
91     for i in range(out_count):
92         print(f"\n--- 输出 #{i+1} ---")
93         # 金额
94         value_sat = int.from_bytes(data[offset:offset+8], 'little')
95         print(f"  金额 (Value): {value_sat} 聪 ({value_sat/1e8:.8f} BTC)")
96         print_bytes_hex_and_bin(data, offset, 8, f"输出#{i+1} 金额字节")
97         offset += 8
98
99         # 锁定脚本长度
100        script_len, s_len = read_varint(data, offset)
101        print(f"  锁定脚本长度 (ScriptPubKey Length): {script_len}")
102        print_bytes_hex_and_bin(data, offset, s_len, f"输出#{i+1} 脚本长度
VarInt")
103        offset += s_len
104        # 锁定脚本
105        script_pubkey = data[offset:offset+script_len]
106        print(f"  锁定脚本 (ScriptPubKey): {script_pubkey.hex()}")
107        if script_len > 0:
108            print_bytes_hex_and_bin(data, offset, script_len, f"输出#{i+1} 锁定
脚本字节")
109        else:
110            print("    (空脚本)")
111        offset += script_len
112
113        # 6. 锁定时间
114        locktime = int.from_bytes(data[offset:offset+4], 'little')
115        print(f"\n4. 锁定时间 (LockTime): {locktime}")
116        print_bytes_hex_and_bin(data, offset, 4, "锁定时间字节")
117        offset += 4
118
119        if offset == len(data):
120            print("\n解析完成, 所有字节已读尽! ")
121        else:
122            print(f"\n剩余 {len(data)-offset} 个字节未解析")
123
124    if __name__ == "__main__":
125        # 从文件 raw_tx.hex 读取交易十六进制
126        with open('raw_tx.hex', 'r') as f:
127            hex_str = f.read().strip()
128            parse_transaction(hex_str)
129

```



```

主 页  我的计算机  靶机  Ubuntu 64 位 (2)
Activities Terminal
Jul 28 22:02
ubuntu@ubuntu: ~/Desktop/bitcoin_homework

交易原始数据解析
总字节数: 225

1. 版本号 (Version): 1 (0x00000001)
版本号字节 (长度 4 字节):
[ 0] 0x01 00000001
[ 1] 0x00 00000000
[ 2] 0x00 00000000
[ 3] 0x00 00000000

2. 输入计数 (Input Count): 1
输入计数 VarInt 字节 (长度 1 字节):
[ 0] 0x01 00000001

--- 输入 #1 ---
前序交易哈希 (Previous Tx Hash): 78468ee6981ae1465998122fbd3e95284d72280e7773b61b0b9aefc36f12fd04
输入#1 前序哈希字节 (长度 32 字节):
[ 0] 0x04 00000100
[ 1] 0xf0 11111101
[ 2] 0x12 00010010
[ 3] 0xf6 01101111
[ 4] 0xc9 11001001
[ 5] 0xef 11101111
[ 6] 0x9a 10011010
[ 7] 0xb0 01101011
[ 8] 0x1b 00011011
[ 9] 0xb6 10110110
[10] 0xf3 01110011
[11] 0x77 01101111
[12] 0xe6 00001110
[13] 0x20 00100000
[14] 0x72 01110010
[15] 0x4d 01001101
[16] 0x28 00101000
[17] 0x95 10010101
[18] 0x3e 00111110
[19] 0xbd 10111101
[20] 0x2f 00101111
[21] 0x12 00010010
[22] 0x98 10011000
[23] 0x59 01011001
[24] 0x46 01000110
[25] 0xe1 11100001
[26] 0x1a 00011010
[27] 0x90 10010000
[28] 0xe6 11100110
[29] 0x8e 10001110
[30] 0x4a 01000110
[31] 0x78 01111000

前序输出索引 (Previous Index): 1
输入#1 索引字节 (长度 4 字节):
[ 0] 0x01 00000001
[ 1] 0x00 00000000
[ 2] 0x00 00000000
[ 3] 0x00 00000000

解锁脚本长度 (ScriptSig Length): 106
输入#1 脚本长度VarInt (长度 1 字节):
[ 0] 0x6a 01101010

```

```

主 页  我的计算机  靶机  Ubuntu 64 位 (2)
Activities Terminal
Jul 28 22:02
ubuntu@ubuntu: ~/Desktop/bitcoin_homework

解锁脚本长度 (ScriptSig Length): 106
输入#1 脚本长度VarInt (长度 1 字节):
[ 0] 0x6a 01101010

输入#1 解锁脚本字节 (长度 106 字节):
[ 0] 0x47 01000111
[ 1] 0x30 00110000
[ 2] 0x44 01000100
[ 3] 0x02 00000010
[ 4] 0x20 00100000
[ 5] 0x69 01101001
[ 6] 0x40 01000000
[ 7] 0x51 01010001
[ 8] 0x1f 00011111
[ 9] 0x4d 01001101
[10] 0x20 00100000
[11] 0x5b 01011011
[12] 0xad 10101101
[13] 0xe2 11100010
[14] 0x21 00100001
[15] 0xeb 11101011
[16] 0xbc 10111100
[17] 0x09 01101001
[18] 0x3b 00111011
[19] 0x94 10010100
[20] 0xd0 01101101
[21] 0x5f 01011111
[22] 0x6a 01101010
[23] 0x50 01010110
[24] 0xd8 11011000
[25] 0x9d 10011101
[26] 0x22 00100010
[27] 0x9c 10011100
[28] 0xcd 11001101
[29] 0xf3 11110011
[30] 0xf6 11110101
[31] 0xeb 11101011
[32] 0x7f 01111111
[33] 0xfc 11111100
[34] 0x2c 00101100
[35] 0xc3 11000011
[36] 0x06 01100110
[37] 0x02 00000010
[38] 0x20 00100000
[39] 0x77 01101111
[40] 0xd8 11011000
[41] 0xe6 11100110
[42] 0x2b 00101011
[43] 0x80 10000000
[44] 0xf9 11111001
[45] 0xf2 11110010
[46] 0xa4 10100100
[47] 0x5d 01011101
[48] 0xf6 11110110
[49] 0xf7 01101111
[50] 0xe1 11100001
[51] 0x41 01000001
[52] 0x8a 00001010
[53] 0x46 01000110
[54] 0xde 11011110
[55] 0x64 01100100

```

```

主 页  ×  我的计算机  ×  靶机  ×  Ubuntu 64 位 (2)  ×
Activities  Terminal  Jul 28 22:03
ubuntu@ubuntu: ~/Desktop/bitcoin_homework

[56] 0x05 00000101
[57] 0xa5 10100101
[58] 0x4a 01001010
[59] 0xee 11011110
[60] 0xd0 00001101
[61] 0x64 01100100
[62] 0xb0 10110000
[63] 0xbd 10111101
[64] 0x04 00000100
[65] 0x7f 01111111
[66] 0x2a 00101010
[67] 0x3c 00111100
[68] 0x1d 00011101
[69] 0xb9 00001001
[70] 0x4e 01001110
[71] 0x01 00000001
[72] 0x21 00100001
[73] 0x03 00000011
[74] 0xcc 11001100
[75] 0xdc 11011100
[76] 0x2b 00101011
[77] 0x75 01110101
[78] 0xb3 10110011
[79] 0x97 10010111
[80] 0xf6 11110110
[81] 0x44 01000100
[82] 0xda 11011010
[83] 0xd9 11011001
[84] 0x50 01010101
[85] 0x3d 00111101
[86] 0xaf 10101111
[87] 0xa5 10100101
[88] 0x0e 00000110
[89] 0x82 10000010
[90] 0x9c 10011100
[91] 0x85 10000101
[92] 0xdb 11011011
[93] 0x45 01000101
[94] 0xdb 11011011
[95] 0xb7 10111111
[96] 0xef 11101111
[97] 0x50 01100101
[98] 0xca 11001010
[99] 0x5c 01011100
[100] 0xaa 10101010
[101] 0xfa 11111010
[102] 0xd2 11010010
[103] 0xe1 11100001
[104] 0x21 11100001
[105] 0x7c 01111100

序列号 (Sequence): 4294967295
输入#1 序列号字节 (长度 4 字节):
[ 0] 0xff 11111111
[ 1] 0xff 11111111
[ 2] 0xff 11111111
[ 3] 0xff 11111111

3. 输出计数 (Output Count): 2
输出计数 Varint 字节 (长度 1 字节):
[ 0] 0x02 00000010

```

```

主 页  ×  我的计算机  ×  靶机  ×  Ubuntu 64 位 (2)  ×
Activities  Terminal  Jul 28 22:03
ubuntu@ubuntu: ~/Desktop/bitcoin_homework

[ 3] 0xff 11111111

3. 输出计数 (Output Count): 2
输出计数 Varint 字节 (长度 1 字节):
[ 0] 0x02 00000010

--- 输出 #1 ---
金额 (Value): 5000 聪 (0.00005000 BTC)
输出#1 金额字节 (长度 8 字节):
[ 0] 0x08 10001000
[ 1] 0x13 00010011
[ 2] 0x00 00000000
[ 3] 0x00 00000000
[ 4] 0x00 00000000
[ 5] 0x00 00000000
[ 6] 0x00 00000000
[ 7] 0x00 00000000

锁定脚本长度 (ScriptPubkey Length): 25
输出#1 脚本长度Varint (长度 1 字节):
[ 0] 0x19 00011001

锁定脚本 (ScriptPubKey): 76a914a30e7f8c2e439d8e83b1ecd8f8a97dfbc67c90c588ac
[ 7] 0x00 00000000

锁定脚本长度 (ScriptPubkey Length): 25
输出#1 脚本长度Varint (长度 1 字节):
[ 0] 0x19 00011001

锁定脚本 (ScriptPubKey): 76a914a30e7f8c2e439d8e83b1ecd8f8a97dfbc67c90c588ac
输出#1 锁定脚本字节 (长度 25 字节):
[ 0] 0x76 01101010
[ 1] 0xa9 10101001
[ 2] 0x14 00010100
[ 3] 0xa3 10100011
[ 4] 0x0e 00001110
[ 5] 0x7f 01111111
[ 6] 0x08 10001000
[ 7] 0x2e 00101110
[ 8] 0x43 01000011
[ 9] 0x9d 10011101
[10] 0x0e 10001110
[11] 0x83 10000011
[12] 0xb1 10110001
[13] 0xec 11101100
[14] 0xd8 11011000
[15] 0xf6 11111000
[16] 0xa9 10101001
[17] 0x7d 01111101
[18] 0xfb 11111011
[19] 0xcc 11000110
[20] 0x7c 01111100
[21] 0x90 10010000
[22] 0xc5 11000101
[23] 0x88 10001000
[24] 0xac 10101100

--- 输出 #2 ---
金额 (Value): 55204 聪 (0.00055204 BTC)

```

```

[16] 0xa9 10101001
[17] 0x7d 01111011
[18] 0xfb 11111011
[19] 0xc6 11000110
[20] 0x7c 01111000
[21] 0x90 10010000
[22] 0xc5 11000101
[23] 0x88 10001000
[24] 0xac 10101100

--- 输出 #2 ---
会集 (value): 85204 聪 (0.0085204 BTC)
输出#2: 会集字节 (长度 8 字节):
[ 0] 0xa4 10100100
[ 1] 0xd7 11010111
[ 2] 0x00 00000000
[ 3] 0x00 00000000
[ 4] 0x00 00000000
[ 5] 0x00 00000000
[ 6] 0x00 00000000
[ 7] 0x00 00000000

锁定脚本长度 (ScriptPubKey Length): 25
输出#2: 脚本长度VarInt (长度 1 字节):
[ 0] 0x19 00011001

锁定脚本 (ScriptPubKey): 76a914759c1790f092a213dd2e2f2cbc4ec1544a1e9e688ac
输出#2: 锁定脚本字节 (长度 25 字节):
[ 0] 0x76 01101011
[ 1] 0xa9 10101001
[ 2] 0x14 00010100
[ 3] 0x7c 01110101
[ 4] 0x9c 10011100
[ 5] 0x17 00010111
[ 6] 0x90 10010000
[ 7] 0xf0 11100000
[ 8] 0x92 10010010
[ 9] 0xa2 10100010
[10] 0x13 00010011
[11] 0xdd 11011101
[12] 0x2e 00101110
[13] 0x2e 00101110
[14] 0xf2 11110010
[15] 0xcb 11001011
[16] 0xc4 11000100
[17] 0xec 11101100
[18] 0x15 00010101
[19] 0x44 01000100
[20] 0xa1 10100001
[21] 0xe9 11101001
[22] 0xe6 11100110
[23] 0x88 10001000
[24] 0xac 10101100

4. 锁定时间 (lockTime): 0
输出时间字节 (长度 4 字节):
[ 0] 0x00 00000000
[ 1] 0x00 00000000
[ 2] 0x00 00000000
[ 3] 0x00 00000000

```

然后获取完整区块数据：

创建 get_raw_block.py：

代码块

```

1  import requests
2
3  # 获取最新区块哈希
4  tip_url = 'https://blockstream.info/testnet/api/blocks/tip/hash'
5  block_hash = requests.get(tip_url).text.strip()
6  print("最新区块哈希:", block_hash)
7
8  # 获取该区块的原始十六进制数据
9  raw_url = f'https://blockstream.info/testnet/api/block/{block_hash}/raw'
10 block_hex = requests.get(raw_url).text.strip()
11
12 # 保存到文件
13 with open('raw_block.hex', 'w') as f:
14     f.write(block_hex)
15
16 print(f"区块原始数据已保存到 raw_block.hex")
17 print(f"区块大小: {len(block_hex)//2} 字节")
18

```

```
(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$ nano get_raw_block.py
(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$ python3 get_raw_block.py
最新区块哈希: 0000000000000000d0c7d8027a5a0382f4ed6145eb47586908c92cf1aeadf3db8
区块原始数据已保存到 raw_block.hex
区块大小: 895 字节
```

代码块

```
1 0000000000000000d0c7d8027a5a0382f4ed6145eb47586908c92cf1aeadf3db8
2
```

然后解析区块头并验证PoW:

创建parse_block.py

代码块

```
1 import hashlib
2 import time
3 import sys
4
5 def double_sha256(data):
6     return hashlib.sha256(hashlib.sha256(data).digest()).digest()
7
8 def read_varint(data, offset):
9     first = data[offset]
10    if first < 0xFD:
11        return first, 1
12    elif first == 0xFD:
13        return int.from_bytes(data[offset+1:offset+3], 'little'), 3
14    elif first == 0xFE:
15        return int.from_bytes(data[offset+1:offset+5], 'little'), 5
16    elif first == 0xFF:
17        return int.from_bytes(data[offset+1:offset+9], 'little'), 9
18    else:
19        raise ValueError("Invalid VarInt")
20
21 def parse_block(block_hex):
22     data = bytes.fromhex(block_hex)
23     print(f"区块总字节数: {len(data)}\n")
24
25     offset = 0
26     # ---- 区块头 ----
27     header = {}
28     header['version'] = int.from_bytes(data[offset:offset+4], 'little')
29     offset += 4
30     header['prev_hash'] = data[offset:offset+32][::-1].hex()
```

```

31     offset += 32
32     header['merkle_root'] = data[offset:offset+32][::-1].hex()
33     offset += 32
34     header['timestamp'] = int.from_bytes(data[offset:offset+4], 'little')
35     offset += 4
36     header['bits'] = int.from_bytes(data[offset:offset+4], 'little')
37     offset += 4
38     header['nonce'] = int.from_bytes(data[offset:offset+4], 'little')
39     offset += 4
40
41     print("=== 区块头 (80字节) ===")
42     for k, v in header.items():
43         if k == 'timestamp':
44             print(f"    {k}: {v} ({time.ctime(v)})")
45         else:
46             print(f"    {k}: {v}")
47
48     # 计算区块哈希
49     block_hash = double_sha256(data[:80])[::-1].hex()
50     print(f"\n计算的区块哈希: {block_hash}")
51
52     # PoW 验证
53     exponent = (header['bits'] >> 24) & 0xFF
54     coefficient = header['bits'] & 0xFFFFFF
55     target = coefficient * (256 ** (exponent - 3))
56     hash_int = int.from_bytes(double_sha256(data[:80]), 'little')
57     print(f"目标难度值 (Target): {target}")
58     print(f"哈希整数: {hash_int}")
59     if hash_int <= target:
60         print("PoW 验证通过!")
61     else:
62         print("PoW 验证失败")
63
64     # ---- 交易列表 ----
65     tx_count, v_size = read_varint(data, offset)
66     print(f"\n交易数量: {tx_count}")
67     offset += v_size
68
69     print("\n=== 交易列表 ===")
70     for i in range(tx_count):
71         print(f"交易 #{i+1}: 偏移量 {offset}")
72         offset += 1
73
74
75 if __name__ == "__main__":
76     with open('block_hex.txt', 'r') as f:
77         hex_str = f.read().strip()

```

```
78     parse_block(hex_str)
```

```
79
```

```
80
```

```
(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$ python3 parse_block.py
```

区块总字节数: 1423

=== 区块头 (80字节) ===

version: 662118400

prev_hash: 000000000000017b2ab1ac42fb199920cb3ae2869619237c148378930987b70c

merkle_root: da1b13434a55542b8572891e50dca4480a60937b881a77970c968ea981b3837c

timestamp: 1785249269 (Tue Jul 28 07:34:29 2026)

bits: 436694572

nonce: 3242643830

计算的区块哈希: 0000000000000001b0ead5379548313f248b94fce0fbc6edab3ae4aef0317ef8e

目标难度值 (Target): 11940126377566236398571955759248095341091947647712610899460096

哈希整数: 169841627195940918635132685021691408362090239760748068204430

PoW 验证通过!

交易数量: 6

=== 交易列表 ===

交易 #1: 偏移量 81

交易 #2: 偏移量 82

交易 #3: 偏移量 83

交易 #4: 偏移量 84

交易 #5: 偏移量 85

交易 #6: 偏移量 86

```
(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$
```

然后解析解析脚本操作码:

创建parse_script.py

代码块

```
1  OP_CODES = {
2      0x00: 'OP_0',
3      0x76: 'OP_DUP',
4      0xa9: 'OP_HASH160',
5      0x88: 'OP_EQUALVERIFY',
6      0xac: 'OP_CHECKSIG',
7      0x6a: 'OP_RETURN',
8  }
9
10 def parse_script(script_hex):
11     data = bytes.fromhex(script_hex)
12     i = 0
13     result = []
14     while i < len(data):
15         op = data[i]
16         if op in OP_CODES:
17             result.append(OP_CODES[op])
```

```

18         i += 1
19         elif 0x01 <= op <= 0x4b:
20             length = op
21             data_push = data[i+1:i+1+length].hex()
22             result.append(f"PUSH {length} bytes: {data_push}")
23             i += 1 + length
24         else:
25             result.append(f"UNKNOWN(0x{op:02x})")
26             i += 1
27         return result
28
29 if __name__ == '__main__':
30     script = input("请输入脚本十六进制: ")
31     ops = parse_script(script)
32     for op in ops:
33         print(op)

```

```

(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$ python3 parse_script.py
请输入脚本十六进制: 47304402206940511f4d205bade221ebbc693b946d5f6a56d89d229ccd
f3f5eb7ffc2cc366022077d8e62b80f9f2a45df66fe1410a46de6405a54aee0d64b0bd047f2a3c
1d094e012103ccdc2b75b397f644dad9553dafa506829c85db45dbbfef65ca5caafad2e1e17c
PUSH 71 bytes: 304402206940511f4d205bade221ebbc693b946d5f6a56d89d229ccdf3f5eb7
ffc2cc366022077d8e62b80f9f2a45df66fe1410a46de6405a54aee0d64b0bd047f2a3c1d094e0
1
PUSH 33 bytes: 03ccdc2b75b397f644dad9553dafa506829c85db45dbbfef65ca5caafad2e1e
17c
(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$ python3 parse_script.py
请输入脚本十六进制: 76a914a30e7f8c2e439d8e83b1ecd8f8a97dfbc67c90c588ac
OP_DUP
OP_HASH160
PUSH 20 bytes: a30e7f8c2e439d8e83b1ecd8f8a97dfbc67c90c5
OP_EQUALVERIFY
OP_CHECKSIG
(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$ python3 parse_script.py
请输入脚本十六进制: 76a914759c1790f092a213dd2e2ef2cbc4ec1544a1e9e688ac
OP_DUP
OP_HASH160
PUSH 20 bytes: 759c1790f092a213dd2e2ef2cbc4ec1544a1e9e6
OP_EQUALVERIFY
OP_CHECKSIG
(base) ubuntu@ubuntu:~/Desktop/bitcoin_homework$

```

第一次运行的是解锁脚本：

代码块

```

1  PUSH 71 bytes:
    304402206940511f4d205bade221ebbc693b946d5f6a56d89d229ccdf3f5eb7ffc2cc366022077d
    8e62b80f9f2a45df66fe1410a46de6405a54aee0d64b0bd047f2a3c1d094e01

```

```
2  PUSH 33 bytes:
    03ccdc2b75b397f644dad9553dafa506829c85db45dbbfe65ca5caafad2e1e17c
```

这是P2PKH 解锁脚本，包含：

71字节的 DER 签名

33字节的公钥

第二次运行的是锁定脚本（输出 #1）

代码块

```
1  OP_DUP
2  OP_HASH160
3  PUSH 20 bytes: a30e7f8c2e439d8e83b1ecd8f8a97dfbc67c90c5
4  OP_EQUALVERIFY
5  OP_CHECKSIG
```

这是标准的 P2PKH 锁定脚本，含义：

OP_DUP：复制栈顶公钥

OP_HASH160：对公钥进行 RIPEMD-160 哈希

PUSH 20 bytes：推送公钥哈希（接收方地址的哈希）

OP_EQUALVERIFY：检查栈顶两个哈希是否相等

OP_CHECKSIG：验证签名是否有效

第三次运行的是锁定脚本（输出 #2，找零）

代码块

```
1  OP_DUP
2  OP_HASH160
3  PUSH 20 bytes: 759c1790f092a213dd2e2ef2cbc4ec1544a1e9e6
4  OP_EQUALVERIFY
5  OP_CHECKSIG
```

结构与第二次完全相同，只是公钥哈希不同

实验总结：

本次实验成功完成了以下任务：

在测试网生成钱包、获取测试币并成功发送交易；

获取交易的原始十六进制，并逐字节解析到每个比特位；

获取一个完整区块，解析了80字节区块头的所有字段；

验证了PoW，确认区块哈希满足难度目标；

1. 解析了脚本操作码，理解了锁定脚本与解锁脚本的工作方式。